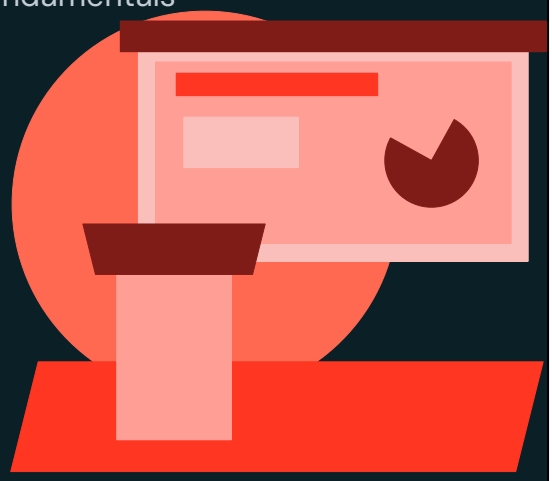
databricks

Software Engineering, DevOps, and CI/CD Fundamentals

LECTURE

Introduction to Modularizing PySpark Code



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

In this lecture, we will discuss how to modularize PySpark code and explore the benefits of doing so.

Modularizing PySpark Code

Non-Modularized Code (Before)

```
# Load data
df = (spark
     .read
     .csv("health.csv",
          header=True,
          inferSchema=True))

# Create column
df = (
 df
 .withColumn("NewColumn",
  when(col("Column") == 0, 'Normal')
  .otherwise('Unknown'))
)
```

Issues

- Everything is in **one block**, making it harder to modify or test specific parts (e.g., loading data, adding new columns).
- Code **duplication** could occur if the same operations are needed elsewhere in the project.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's begin by taking a look at some non-modularized code. Looking at the example here, who has written code like this before? Where code is split into several lines or cells, consistently writing specific logic to transforms or prepare or analyze your data? As you continue to build your flow the code becomes hard to read and unmanageable? My guess is, we've all been there.

While this approach can work, it creates several challenges when trying to implement CI/CD into your development process.

First, everything is lumped into one block, making it harder to modify or test specific parts of the code (for example, loading data or adding new columns).

Another issue is code duplication. If the same operations are needed elsewhere in the project, you might end up repeating yourself, leading to unnecessary complexity and maintenance headaches. When one thing changes in your logic, you have to change that everywhere.

Modularizing PySpark Code

Modularized Code (After)

```python
# Load data
df = (spark
     .read
     .csv("health.csv",
          header=True,
          inferSchema=True))
```

```python
def load_data(file_path):
    return (spark
            .read
            .csv(file_path,
                 header=True,
                 inferSchema=True))
```

```python
# Create column
df = (
 df
 .withColumn("NewColumn",
  when(col("Column") == 0, 'Normal')
  .otherwise('Unknown'))
)
```

```python
def add_new_col(df, new, s_col):
    return (df
            .withColumn(new,
               when(col(s_col) == 0, 'Normal')
               .otherwise('Unknown')))
```

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Instead, you want to focus on modularizing your code.

For instance, the code you wrote to read the CSV file into a Spark DataFrame can be turned into a function, such as def load_data().

Similarly, the function that creates a new column based on an existing one could be refactored into def add_new_col(), or, ideally, a more specific name that clearly reflects what the function is doing (example, def add_age_category() if you're categorizing ages).

Modularizing PySpark Code

Modularized Code Benefits

Benefits

- **Easier Maintenance** by updating only specific functions without changing the entire script.
- **Reuse** functions in different projects.
- **Testing** individual functions through unit tests to ensure code reliability.

```
def load_data(file_path):
    return (spark
            .read
            .csv(file_path,
                 header=True,
                 inferSchema=True))


def add_new_col(df, new, s_col):
    return (df
            .withColumn(new,
               when(col(s_col) == 0, 'Normal')
               .otherwise('Unknown')))
```

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Let's talk about some of the key benefits of modularizing you code.

First, functions make maintenance easier. When your code is organized into functions, you can update or change specific parts of the code without affecting the entire script. This makes it way easier to manage and fix issues down the line.

Another big benefit is reusability. Once you've written a function like load_data() or add_new_col(), you can reuse it across different projects or scenarios. No need to rewrite the same code over and over again. it saves time and reduces errors.

Lastly, functions improve testability. By isolating specific logic into functions, you can write unit tests for each function individually, ensuring that each part of your code is working as expected. This leads to more reliable and bug-free code.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Thank you for completing this lesson and continuing your journey to develop your skills with us.